

001 Reference Architecture

 Page Status - Complete

Page Description

This page is designed to document the reference architecture of the enterprise data platform.

Table of Contents

- [Page Description](#)
- [Table of Contents](#)
- [What is an Enterprise Data Platform \(EDP\)?](#)
- [Reference Architecture](#)
- [Data Sources and Extracts](#)
- [Ingress Reference Patterns](#)
- [Data Lake](#)
- [Semantic Layer](#)
- [Non-Functional Requirements](#)
- [Attachments](#)

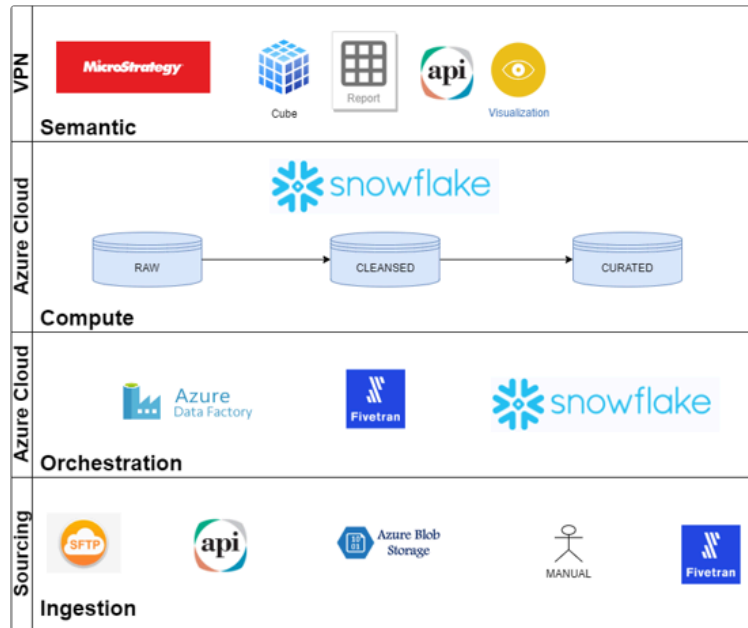
What is an Enterprise Data Platform (EDP)?

An Enterprise Data Platform (EDP) is a central location in an organization where data is:

- Collected
- Organized
- Curated

The reference architecture diagram identifies the components of an Enterprise Data Architecture and application stack(s) employed for the component

Reference Architecture



Data Sources and Extracts

Data sources are identified for use in EDP based on requirements, business capabilities and business benefits. Each data source is researched for its usability, confidence in consistent receipt (ingress reference pattern) and appropriate application for the capabilities offered in EDP. List of current and future data sources and their specific interfaces is provided in the link below.

Ingress Reference Patterns

Ingress reference patterns are the means and protocol available for receiving data. The ingress patterns define a common handshake for data interchange implemented by a specific resource (ex Mulesoft, Azure Data Factory) or protocol (ex sFTP). The patterns are documented for a generic use case then applied based on an approved High Level Design (HLD) document which details the specific network and protocols.

Data Lake

A data lake is the implementation of an organized information storage design with the intent of providing data for business decisions, exploration, reporting and tool/application data consumption. It is both a repository for information in the company and the application of data standards designed for consistent implementation and ease of use.

The Data Lake can be defined by its Resources & Standards and Designs

- Resources: Employed components in the system for storage, monitoring, orchestration and modelling
- Standards and Designs: Application of design principles within the Resources

The specifics of the Resource and Standard and Designs are located in the Azure Cloud and Data Lake pages.

Semantic Layer

- The semantic layer represents the design and implementation of business logic and processes to support reporting and discovery.
- Implementation requires understanding how business processes and data relate and connect for consistent and organized consumption.
- Presentation and Data Discovery Layer: The presentation and discovery layer is an application or service available for interacting with data from the semantic or data lake.

Non-Functional Requirements

- Non-Functional requirements define system attributes that serve as constraints or restrictions on the design of the system.
- Each should specify the operation, not behavior of a system.
-

Requirement	Description
Security & Privacy	Security refers to the ability to prevent and/or forbid access to the system by unauthorized usage. Security is ensuring access to information and control to authorized systems, people, and processes as per established business policies. Security and confidentiality requirements should cover all the security related aspects on the system being developed and the project such as : Consider the legal, regulatory and client policy compliance requirements. Consider application, data, platform, and infrastructure security requirements. These must be clearly identified including the functional security requirements when addressing authentication, authorization, confidentiality, integrity, availability and accountability, cookies, encryption, access controls (including role based/segregation of duties/maker-checker principle), auditability (related to regulatory or legal requirements such as SOX). Consider secure development

	lifecycle requirements :These include threat and vulnerability assessment requirements, security reviews of design and code, etc.
Performance Security	Review of persona and access with a view to legal, monetary and personal
Scalability	Scalability is the ability for a system to grow and meet performance goals with an increased volume of activity and data. Software that is scalable has the ability to handle a wide variety of system configuration sizes. The nonfunctional requirements should specify the ways in which the system may be expected to scale up or out (by increasing hardware capacity, adding machines, etc.) Scaling up or out, should not require changes in the design of the deployment architecture or loss of service due to the time required to add additional resources. It boils down to how the system copes (or would cope) with additional demand. The main factors that needs to be considered while capturing these requirements are:- User volume Requirement Description Transaction Volume Data Size and Volume Database and File sizing Growth in data Growth in users
Capacity	Capacity refers to the ability of the system to handle transactional volumes . Need to consider existing transaction volume, expected growth and concurrent users.
Availability	A system's availability (uptime) is the amount of time that it is operational and available for use. It is the amount of time that the system is up and running for the users to perform their operations. This is specified because some systems are designed with expected downtime for activities like database upgrades and backups. The availability of a system is the probability that it will be operational when it is needed, which includes Reliability. The more reliable a system is, the more available the system will be. When creating availability requirements, typical questions include What is the application's expected availability in operation time. What is the application's expected availability in a percentage of time. Ensure that systems are running in accordance to business requirements and needs Enable hardware and software redundancy Enable active/standby Downtime Upgrades Minimize Latency
Reliability	Reliability specifies the capability of the software to maintain its performance over time and perform its required functions under stated conditions for a specific period of time. Unreliable software fails frequently, and certain tasks are more sensitive to failure (for example, because they cannot be restarted, or because they must be run at a certain time). Constraints on the run-time behavior of the system Can be considered under two separate headings: Availability - is the

	system available for service when requested by end-users. Failure rate - how often does the system fail to deliver the service as expected by end-users. Many related items: Dependability - Dependability of a computing system is the ability to deliver service that can justifiably be trusted by users Reputation - the general opinion (more technically, a social evaluation) of the public toward a person, a group of people
Recoverability	Recoverability is the ability of a system to resume normal operation after failures/outages occur. It is the application's ability to function correctly under both normal and abnormal operating conditions. A key aspect of recoverability is for a system to maintain data integrity after any sort of abnormal termination. The data that the application may be processing should be recovered and put back online within a reasonable amount of time. Both Recovery Point Objective (RPO) and Recovery Time Objective (RTO) should be captured through these requirements.
Maintainability	Maintainability is defined as the ability to perform successful repair actions within acceptable time, cost, and quality constraints. In other words, maintainability measures the ease and speed with which a system can be restored to operational status after a failure occurs or an enhancement is made. Maintainability has to do with how easy it is to understand, fix, test, maintain, enhance, and port/migrate the application. The ease with which a software system or component can be modified or correct faults, improve performances or other attributes or adapt to changed environment: Conceptual Simplicity - What the software does Interface Simplicity - How to use the software Implementation Simplicity - How the software does it
Serviceability	Operability relates to the ability of an Architecture to allow operations to be performed accurately and consistently, at the desired cost, and with the expected impacts. It is the ability to keep a system in a normal operating condition, meeting the pre-defined operational requirements. The requirements should cover all the required elements that need to be considered to implement the system and to integrate with the exiting IT operation tools (monitoring, scheduling, installation/deployment automation, test automation including regression testing, operating guides, etc.). Both Development and Run aspects should be considered when defining the requirements.
Security & Privacy	Security refers to the ability to prevent and/or forbid access to the system by unauthorized usage. Security is ensuring access to information and control to

	authorized systems, people, and processes as per established business policies. Security and confidentiality requirements should cover all the security related aspects on the system being developed and the project such as : Consider the legal, regulatory and client policy compliance requirements. Consider application, data, platform, and infrastructure security requirements. These must be clearly identified including the functional security requirements when addressing authentication, authorization, confidentiality, integrity, availability and accountability, cookies, encryption, access controls (including role based/segregation of duties/maker-checker principle), auditability (related to regulatory or legal requirements such as SOX). Consider secure development lifecycle requirements :These include threat and vulnerability assessment requirements,security reviews of design and code, etc.
Simplicity	The ability of a system or component to perform its required functions under stated conditions for a specific period of time. Common measurements include: Error prevention Fault tolerance Requirement Description Fault detection Fault removal Measurements to maximize reliability Defined, monitored and enforced SLA Interoperability Does solution require integration Componentization How well does the solution isolate cohesive capabilities into clear components or modules
Interoperability	Does solution require integration Componentization How well does the solution isolate cohesive capabilities into clear components or modules
Adaptability	Does extensibility of solution meet requirements Usability Is there business usage acceptance?Speed to Benefit How long does it take to receive business benefit?
Total Cost of Onwership (TCO)	How much does it cost in labor, materials, maintenance to provide business benefit

Attachments

File	Modified
›  reference_architecture.drawio-20240209-204151.png	Feb 09, 2024 by Jeff Scovill
›  reference_architecture.drawio.png	Feb 09, 2024 by Jeff Scovill

 Drag and drop to upload or [browse for files](#) 

 [Download All](#)